Course: DATS 6312
Instructor: Dr. Amir Jafari
Authors: Venkatesh Nagarjuna, Jeongmin An, Manoj Kodihalli

NLP Final Project Proposal

Speculative Decoding for LLM Inference Acceleration on Qwen3

Problem Selection

Large language models generate text autoregressively, producing one token per forward pass. This makes inference fundamentally memory-bandwidth bound: GPU compute sits idle while weights load from memory each step. Speculative decoding addresses this by pairing a small, fast draft model with the full target model. The draft proposes multiple candidate tokens ahead; the target verifies them in a single parallel forward pass using rejection sampling, producing up to several tokens per step without changing output quality. We selected this problem because:

1. it is now a production-standard technique deployed in Google AI Overviews, vLLM, and SGLang,
2. most published benchmarks use flagship GPUs (A100/H100), leaving practitioners on mid-tier hardware like the NVIDIA A10G without concrete deployment guidance, and
3. it directly engages core NLP concepts (autoregressive decoding, probability distributions, sampling strategies) while being systems-oriented and novel for a course project.

We use the recently released Qwen3 model family (April 2025) as our model backbone. Qwen3 dense models match Qwen2.5 models at roughly double the parameter count (e.g., Qwen3-8B matches Qwen2.5-14B in capability), giving us a strong quality baseline within our VRAM budget. The same-family tokenizer and architecture across all Qwen3 sizes also maximizes draft-target alignment for speculative decoding.

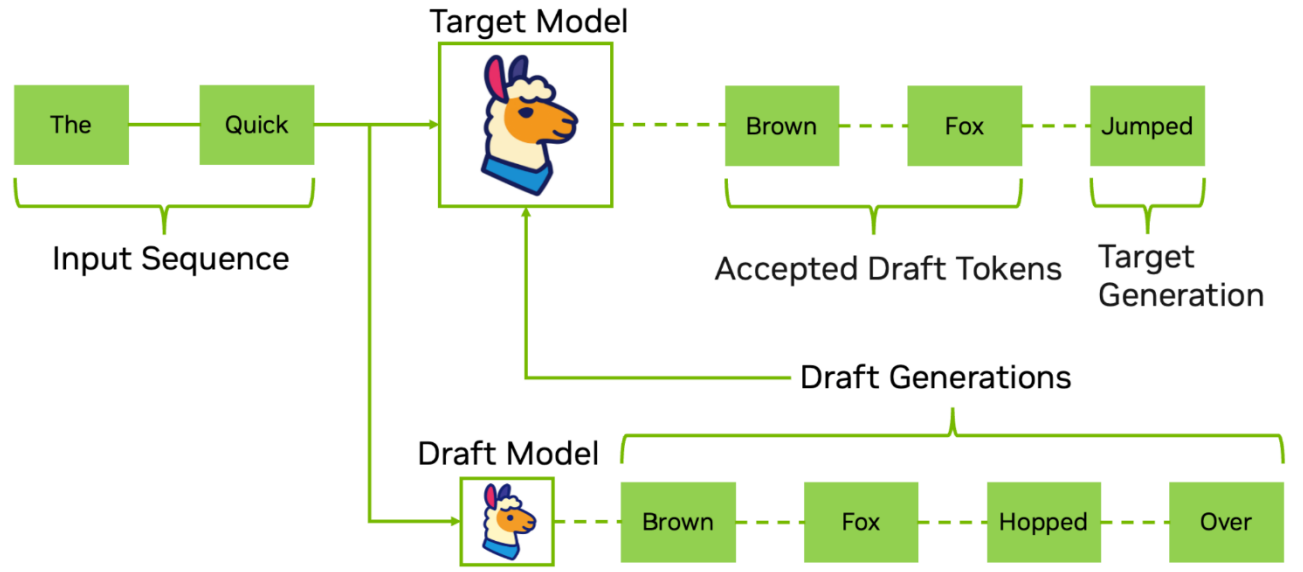

Figure 1: High level overview of speculative decoding using a draft-target system

Datasets

We evaluate speculative decoding across four task domains to measure how acceptance rates vary with output predictability. Our prompt sets are:

- **HumanEval** (code generation, 50 prompts),
- a **TriviaQA** subset (factual QA, 50 prompts),
- **CNN/DailyMail** excerpts (summarization, 50 prompts),
- and **WritingPrompts** (creative writing, 50 prompts).

All are publicly available via HuggingFace Datasets. These span highly structured output (code) to high-entropy output (creative writing), enabling analysis of when speculative decoding helps versus hurts.

NLP Methods and Customization

The core method is speculative sampling with rejection-based verification, as introduced by Leviathan et al. (ICML 2023). This involves autoregressive token generation from a draft model, parallel verification via the target model's forward pass, and stochastic acceptance/rejection based on the ratio of target-to-draft probability distributions. We implement this from scratch rather than using a framework's built-in speculative decoding, giving us full instrumentation over acceptance rates, timing, and per-token statistics. The implementation is customized in that we systematically vary the speculation length (γ), temperature, and draft model size, and measure behavior across task domains. We test three Qwen3 model pair configurations:

(A) Qwen3-8B target + Qwen3-0.6B draft (~20.6 GB, highest alignment baseline),

(B) Qwen3-8B target + Qwen3-1.7B draft (~22.8 GB, tests whether a larger draft's higher accuracy compensates for its greater latency), and

(C) Qwen3-8B target in 4-bit quantization + Qwen3-0.6B draft (~8.2 GB, tests whether target quantization degrades verification quality). Qwen3's thinking mode is disabled for all experiments to ensure standard autoregressive behavior.

Packages

- **PyTorch** for model inference and GPU-accurate timing via CUDA events;
- **HuggingFace Transformers (≥4.51.0)**, required for Qwen3 support, for model loading and tokenization;
- **BitsAndBytes** for 4-bit NF4 quantization of the Qwen3-8B target in Pair C;
- **HuggingFace Datasets** for prompt sourcing; and
- **matplotlib/seaborn** for visualization.

We chose HuggingFace Transformers over vLLM's built-in speculative decoding because we need fine-grained per-token instrumentation that production serving frameworks abstract away.

NLP Objectives

The project spans four generative NLP tasks:

- **code generation** (completing function bodies from HumanEval),
- **closed-book question answering** (factual recall from TriviaQA),
- **abstractive summarization** (condensing CNN/DailyMail articles), and

- **creative text generation** (open-ended story continuation).

These serve as evaluation contexts through which we measure speculative decoding performance; acceptance rates are expected to vary significantly by task—highest for structured code, lowest for creative writing.

Evaluation Metrics

We measure the following metrics:

(1) *acceptance rate α*, the per-token probability of draft acceptance;

(2) *acceptance length τ*, mean tokens accepted per verify round;

(3) *wall-clock speedup*, speculative tokens/sec divided by baseline autoregressive tokens/sec;

(4) *time to first token (TTFT)*;

(5) *peak VRAM usage*; and

(6) *draft overhead ratio*, the fraction of each round spent on draft generation versus target verification.

For greedy decoding (temperature = 0), we verify output correctness via exact string match against the baseline. Each configuration runs 50 prompts with 3 warm-up passes, reporting mean, standard deviation, and p95 latency. The full experiment grid covers 3 model pairs × 5 speculation lengths (γ = 1, 3, 5, 7, 10) × 3 temperatures (0.0, 0.6, 1.0) × 4 task domains = 180 configurations.

Tentative Schedule

Week 1 (Apr 7–13)	Environment setup on AWS g5.2xlarge, install dependencies (transformers ≥4.51.0), download Qwen3-8B, Qwen3-1.7B, and Qwen3-0.6B. Implement core speculative decoding loop and baseline autoregressive decoder. Validate correctness on Pair A.
Week 2 (Apr 14–20)	Run full experiment grid: 180 configurations across 3 model pairs, 5 speculation lengths, 3 temperatures, and 4 task domains. Collect all raw metrics (acceptance rate, throughput, TTFT, VRAM).
Week 3 (Apr 21–27)	Analysis: Pareto frontier plots (speedup vs. α), heatmaps (γ × temperature per task), draft-size comparison (0.6B vs. 1.7B), quantization impact analysis. Draft full report (8–12 pages).
Week 4 (Apr 28–30)	Final polish, address internal review feedback, prepare presentation slides, submit.

Hardware

All experiments run on an AWS g5.2xlarge instance: 1× NVIDIA A10G GPU (24 GB VRAM, ~600 GB/s bandwidth), 8 vCPUs, 32 GB system RAM. All three Qwen3 model pair configurations fit within the 24 GB VRAM budget, with Pair C (quantized target) leaving ~16 GB headroom for ablation flexibility.